KLAi

Your first model: baseline model, linear regression and prediction error

How can a computer predict cooling load from load and temperature, and how do we know it is any good?

CML3112 | Civil Engineering and Architecture | Week 8, class 1 of 4 | Tuesday 3 November 2026, 11:00 to 14:00

KLAi | International University of East Africa | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Explain features, target variable, baseline model and prediction error using the building data



Say why some rows must be held out (validation set and test set) while a model trains



Train a linear regression model and a decision tree in Python and compare them with a baseline model

Your three hours: ideas first, then the lab

0:00	10	Recall from Weeks 1 to 7
0:10	35	Concepts, part 1: data, baseline model and prediction error
0:45	30	Concepts, part 2: learning, held-out data and models
1:15	15	Break
1:30	80	Lab: your first model, in the notebook
2:50	10	Close and what is assessed

Concepts first, with no laptops. Then a break, then the lab with the notebook open.

Three questions from last time

1. What does data.shape tell you about a table?

2. Why do we clean a copy of the file and keep the original unchanged?

3. What is a missing value, and how does pandas show it?



Answer aloud first. Then we check together.

Answers to the three questions at the start of the class

Question	Short answer
What does data.shape tell you about a table?	It gives the number of rows and then the number of columns.
Why do we clean a copy of the file and keep the original unchanged?	So we can always go back to the raw data and check what we changed.
What is a missing value, and how does pandas show it?	It is an empty cell with no reading. pandas shows it as NaN.

Check each answer against your own. If one surprised you, look back at Weeks 1 to 7 before the lab.

Words we use today

	Word	What it means
	**Feature**	An input column (also called an input variable or predictor) that the model reads to make a prediction. Here: load_pct and ambient_c.
	**Target variable**	The column we want to predict. Here: cooling load in kW. When the target is a class such as yes or no, it is called a label. In Class 2 we make a yes or no label from the target.
	**Baseline model**	The simplest possible model, which every other model must beat. Here it predicts the mean of the training targets for every row.
	**Prediction error (residual)**	For one row: actual value minus predicted value. Later slides combine many errors into one score, the MAE or the RMSE.
	**Training set and validation set**	The training set is the rows the model learns from. The validation set is held-out data: rows kept back while the model trains, used to compare models fairly.

A prediction turns past readings into a number you can act on



Size the cooling plant before a hot week arrives



Spot a building that uses more energy than it should



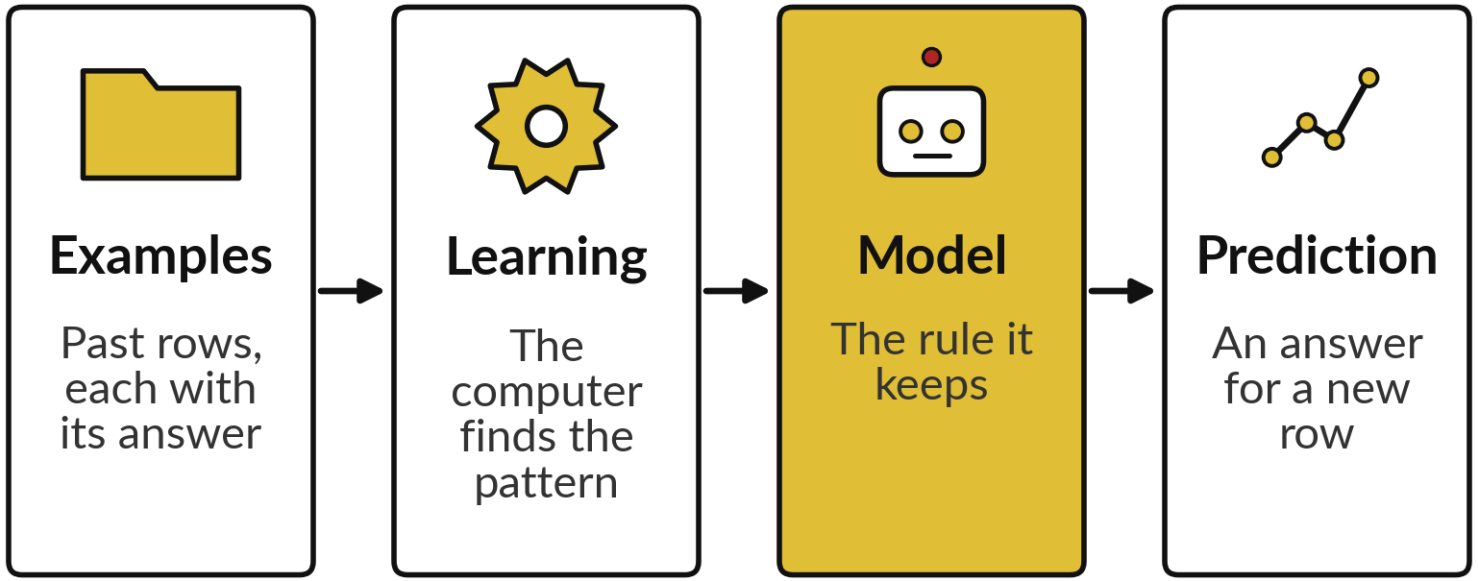
Plan next month’s energy budget

A glossary you will meet again in textbooks and documentation

Term	What it means in plain words
Linear regression	Predicts the target as an intercept plus weighted features. Simple (also called univariate): one feature. Multiple: two or more features. Multivariate regression means several targets, a different thing.
MAE, MSE and RMSE	Three scores that sum up many prediction errors, in the unit of the target (MSE: unit squared).
Missing value and imputation	A missing value is an empty cell (NaN). Imputation fills it, for example with the median.
Parameters and hyperparameter	Parameters (intercept, coefficients) are learned. A hyperparameter, such as max_depth, is chosen before training.
Pipeline	Chains preprocessing and a model, and runs them in order.

Each term is explained again where it first matters.

Machine learning finds the rule from examples, so nobody writes it by hand



A model is the rule the computer found.

We do not write the rule ourselves. The computer finds it from the examples.

Every example row must include its answer, which we call the target variable.

Every row is one reading: the inputs are features and the answer is the target variable

	A	B	C	D	E
	record_id	equipment_id	load_pct	ambient_c	target
1	1	E1	59.15	31.71	52.55
2	2	E2	77.46	29.97	58.69
3	3	E3	69.6	19.87	47.9
4	5	E5	57.62	23.4	45.61

The model reads the features and predicts the target variable (yellow): cooling load in kW.

Features (input variables): load_pct (percent of full load) and ambient_c (deg C).

record_id and equipment_id only label the row.

Record 4 is left out: its target is a missing value (NaN).

Before we model, we remove the rows we cannot trust

file row	record_id	equipment_id	load_pct	ambient_c	target
1	1	E1	59.15	31.71	52.55
4	4	E4	75.98	22.29	NaN
9	9	E3	-5	21.57	57.09
91	1	E1	59.15	31.71	52.55

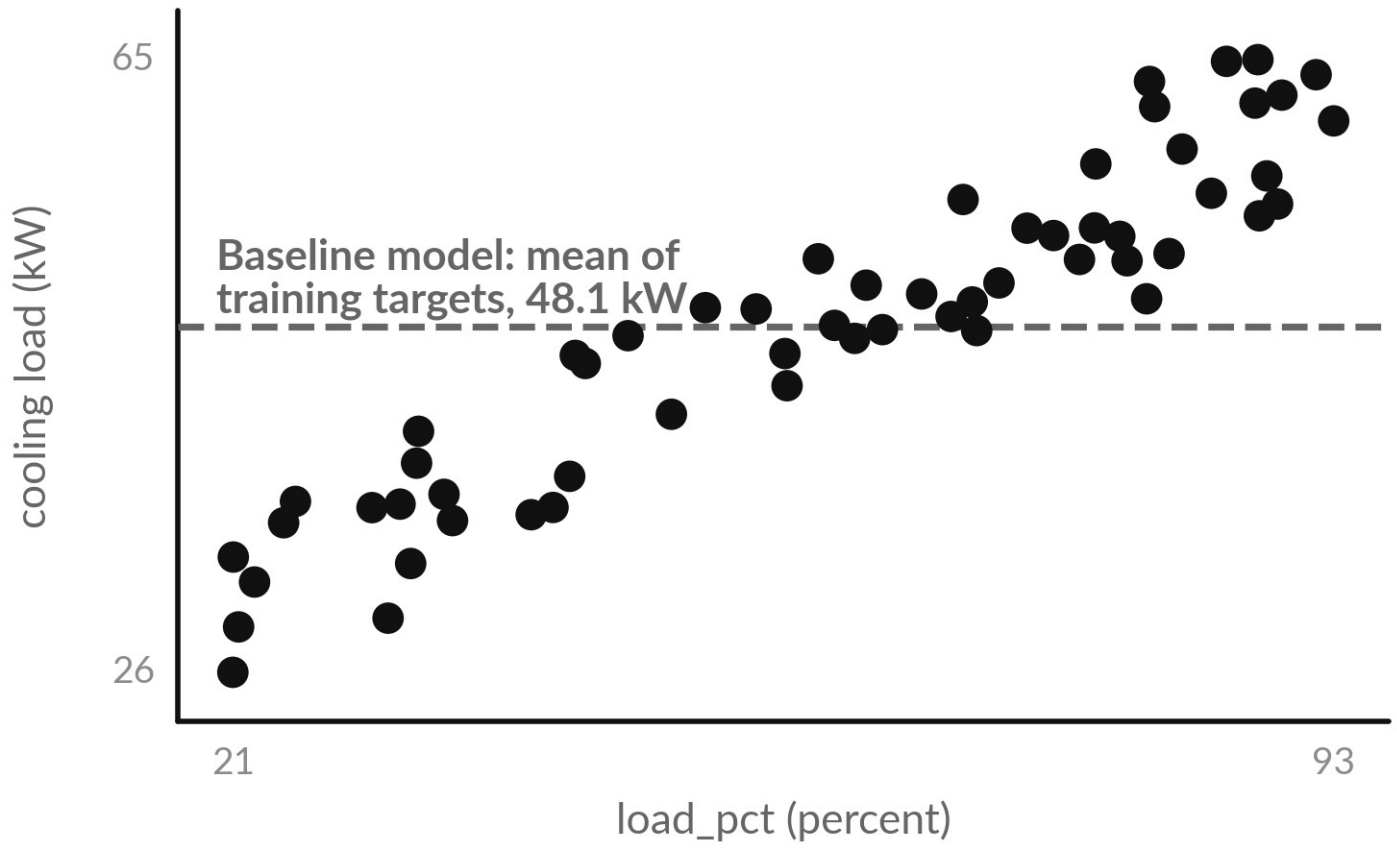
Our file has 91 rows. Three kinds of row cannot be used.

record_id 1 is in row 1 and again in the last row, row 91 (yellow). Keep the first.

record_id 4 has a missing target (NaN): nothing to learn from.

record_id 9 has load_pct -5: impossible.

The baseline model comes first: predict the mean of the training targets every time



Each dot is one training row. The dashed line is the baseline model: it predicts 48.1 kW for every row.

A baseline model is the simplest possible model: it predicts the mean of the training targets and ignores the features.

Every other model must beat it. If it cannot, it has learned nothing useful.

The baseline model as a formula: the mean of the training targets

Baseline model

$$\hat{y} = \bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i$$

Our data

$$\hat{y} = 48.1 \text{ kW}$$

Symbol	What it means	In our data
y-hat and y-bar	The prediction, which is the mean of the y_i.	48.1 kW
y_i	The actual target of training row i.	column target, in kW
n	How many training rows there are.	58 rows (E1 to E4)
sum, i = 1 to n	Add up the values from row 1 to row n.	all training rows

We add the 58 training targets and divide by 58. The validation set plays no part.

Simple linear regression fits a line through the dots and beats the baseline model



Simple (univariate) linear regression: one feature, load_pct. Each extra 1 percent of load adds 0.43 kW.

At zero load it starts at 22.6 kW (the intercept).

It predicts the target as an intercept plus a weighted feature.

On the validation set its MAE is 3.12 kW.

Simple linear regression is a formula with two learned numbers

Simple linear regression

$$\hat{y} = b_0 + b_1 x$$

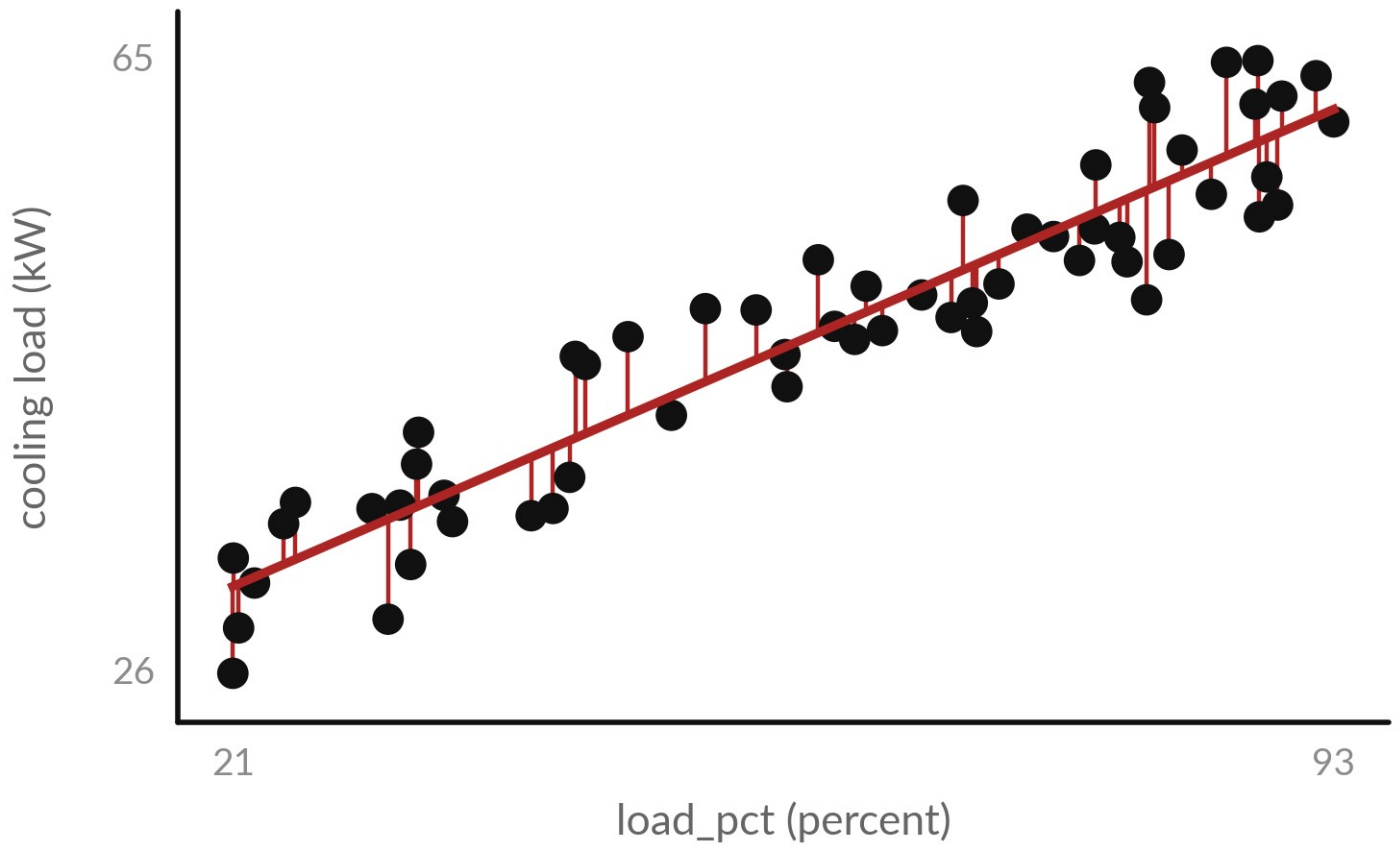
Our fitted model

$$\hat{y} = 22.65 + 0.426\,x$$

Symbol	What it means	In our data
y-hat	The predicted target.	cooling load, in kW
x	The feature of the row. Here the load alone.	column load_pct, in percent
b0	The intercept (bias): the prediction when x is 0.	22.65 kW
b1	The slope, or coefficient (weight): how much the prediction grows when x grows by 1.	0.426 kW per percent of load

Example: at load_pct = 60 the prediction is 22.65 + 0.426 x 60, which is about 48.2 kW.

The prediction error (residual) is the actual value minus the predicted value



Prediction error (residual) = actual value minus predicted value, for one row. The line is simple linear regression.

Each red segment joins a dot (actual) to the line (predicted). Its length is the error size.

Positive error: the model predicted too low.

A good model leaves short segments.

The prediction error of one row is the actual value minus the predicted value

Prediction error of row i

$$e_i = y_i - \hat{y}_i$$

One row of ours

$$e = 45.61 - 48.14 = -2.53 \text{ kW}$$

Symbol	What it means	In our data
e_i	The prediction error (residual) of row i. Negative means the prediction was too high.	in kW
y_i	The actual target of row i.	column target, in kW
y-hat_i	The prediction for row i.	baseline model: 48.14 kW
i and n	The row number i runs from 1 up to n, the number of rows.	one row at a time

Our row: record_id 5, load_pct 57.62, ambient_c 23.4. The baseline model predicted too high by 2.53 kW.

Worked example with hypothetical numbers: four prediction errors give MAE 2.75 and RMSE 3.5

Reading	Actual	Predicted	Error e	Absolute error |e|	Squared error e x e
1	70	72	-2	2	4
2	80	77	+3	3	9
3	90	90	0	0	0
4	60	66	-6	6	36

MAE = (2 + 3 + 0 + 6) / 4 = 2.75. MSE = (4 + 9 + 0 + 36) / 4 = 12.25. RMSE = square root of 12.25 = 3.5. These four readings are hypothetical (made up) to show the arithmetic.

Three ways to turn many errors into one number: MAE, MSE and RMSE

MAE

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|e_i|$$

MSE and RMSE

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}e_i^2 \qquad \text{RMSE} = \sqrt{\text{MSE}}$$

Symbol	What it means	In our data
n	How many rows we score.	15 validation rows
|e_i| and e_i squared	The error with its sign removed, and the error multiplied by itself.	kW and kW squared
sum	Add the values for rows 1 to n.	all validation rows

MAE and RMSE are in the unit of the target. MSE is in the unit squared. Lower is better.

Our own validation rows: the baseline model and simple linear regression

record_id	load_pct	ambient_c	Actual	Baseline model	Baseline error	Linear regression	Linear regression error
5	57.62	23.4	45.61	48.14	-2.53	47.17	-1.56
11	26.14	33.98	36.63	48.14	-11.50	33.77	+2.86
17	61.76	34.28	54.56	48.14	+6.43	48.93	+5.63
23	20.73	30.81	33.52	48.14	-14.61	31.47	+2.05

Values in kW. Linear regression is simple (load_pct only). Absolute errors over 15 rows add up to 145.3 for the baseline (MAE = 145.3 / 15 = 9.69) and 46.9 for linear regression (MAE = 3.12).

MAE and RMSE both measure prediction error in the target's own unit



MAE

The mean of the absolute errors

Every error counts by its size

Easy to explain to a client



RMSE

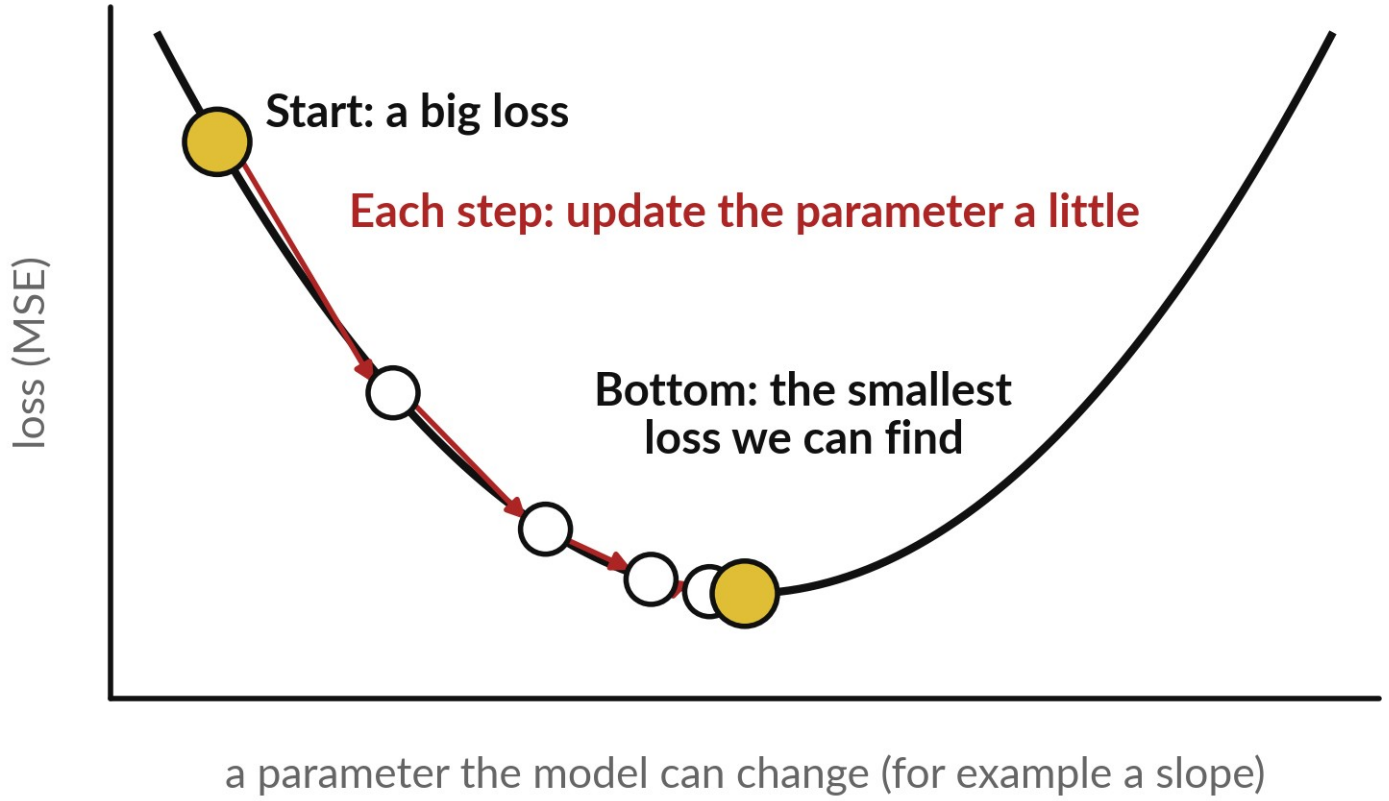
Squares the errors, takes the mean, then the root

One large error counts for much more

Use it when large errors are costly

Both are in kW: "the model is off by about this many kW".

Gradient descent trains a model by walking downhill on the loss, one small step at a time



Training starts with a poor model and improves its parameters (here the slope) by repeating small steps.

Loss function: how wrong the model is, as one number (here the MSE).

Gradient: the slope of the loss. It shows which way is downhill.

Learning rate: the size of each step.

Gradient descent, part 1: how steeply the loss slopes

Slope of the loss

$$\frac{dL}{dw} = \frac{2}{n} \sum_{i=1}^{n} x_i \left(w\, x_i - y_i \right)$$

Symbol	What it means	In our data
L	The loss function: the MSE of the model y-hat = w x.	our tiny illustrative example
w	The parameter (slope) of the model. It has no intercept, to keep the example small.	w stands in for b1
x_i, y_i, n	Input and answer of row i, and the number of rows.	x = 1, 2, 3; y = 2, 4, 6; n = 3
dL/dw	How fast the loss changes when w changes.	at w = 0 it is -18.67

A negative slope of the loss means that a bigger w lowers the loss, so we move w upwards.

Gradient descent, part 2: change w a little, downhill

Update rule

$$w \leftarrow w - \eta \cdot \frac{dL}{dw}$$

Symbol	What it means	In our data
w	The parameter (slope) we are improving. It starts at 0.	the tiny example
arrow	Replace the old w by the new value on the right.	one update per step
dL/dw	The slope of the loss from the last slide.	at w = 0 it is -18.67
eta	The learning rate: the size of each step.	0.05

New w = old w minus the learning rate times the slope of the loss. We repeat this update again and again.

On a tiny illustrative example, five steps almost reach the best slope

Step	w after the step	Loss after the step
0 (start)	0	18.667
1	0.933	5.31
2	1.431	1.51
3	1.697	0.43
4	1.838	0.122
5	1.914	0.035

Illustrative (hypothetical) data: x = 1, 2, 3 and y = 2, 4, 6, so the best w is 2. Step 1 by hand: dL/dw = (2/3) x (1 x (0 - 2) + 2 x (0 - 4) + 3 x (0 - 6)) = -18.67. New w = 0 - 0.05 x (-18.67) = 0.933.

Multiple linear regression has three learned parameters: b0, b1 and b2

Regression function

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2$$

Our learned parameters

$$\hat{y} = 1.49 + 0.447\, x_1 + 0.756\, x_2$$

Symbol	What it means	In our data
y-hat	The predicted target.	cooling load, in kW
x1, x2	The two features (input variables) of a row.	x1 = load_pct (percent), x2 = ambient_c (deg C)
b0	The intercept (bias): the prediction if both features were 0.	1.49 kW
b1, b2	The coefficients (weights) of x1 and x2.	b1 = 0.447 per percent, b2 = 0.756 per deg C

You train this multiple linear regression model in the lab. Training chooses b0, b1 and b2 to make the training MSE small.

What one more unit of a feature does to the prediction

Case	load_pct (percent)	ambient_c (deg C)	Prediction (kW)	Change
Start	60	25	47.18	
load_pct up by 1	61	25	47.63	+0.447
ambient_c up by 1	60	26	47.94	+0.756
Both up by 1	61	26	48.38	+1.202

This is multiple linear regression. Its coefficient for load_pct is not the same as the 0.426 of simple linear regression, because now the two features share the work. The start values 60 and 25 are chosen for the example.

Hold out rows while the model trains, so the evaluation is fair



Training set: the model learns from these rows.

Validation set: we compare models here.

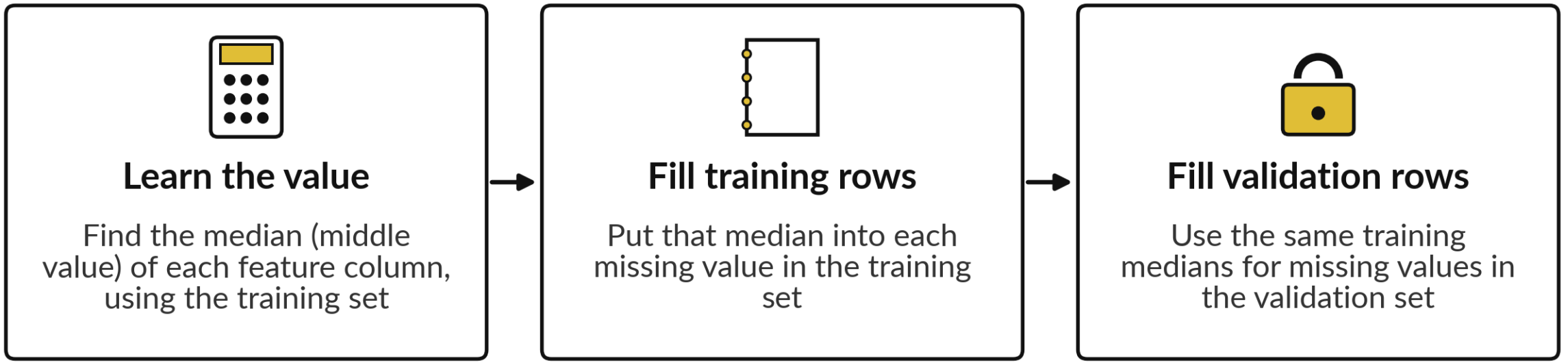
Test set: held out until the end.

After cleaning, 88 rows remain. We cut them into three sets by equipment group.

Held-out data are rows not used to fit the model: the validation and test sets.

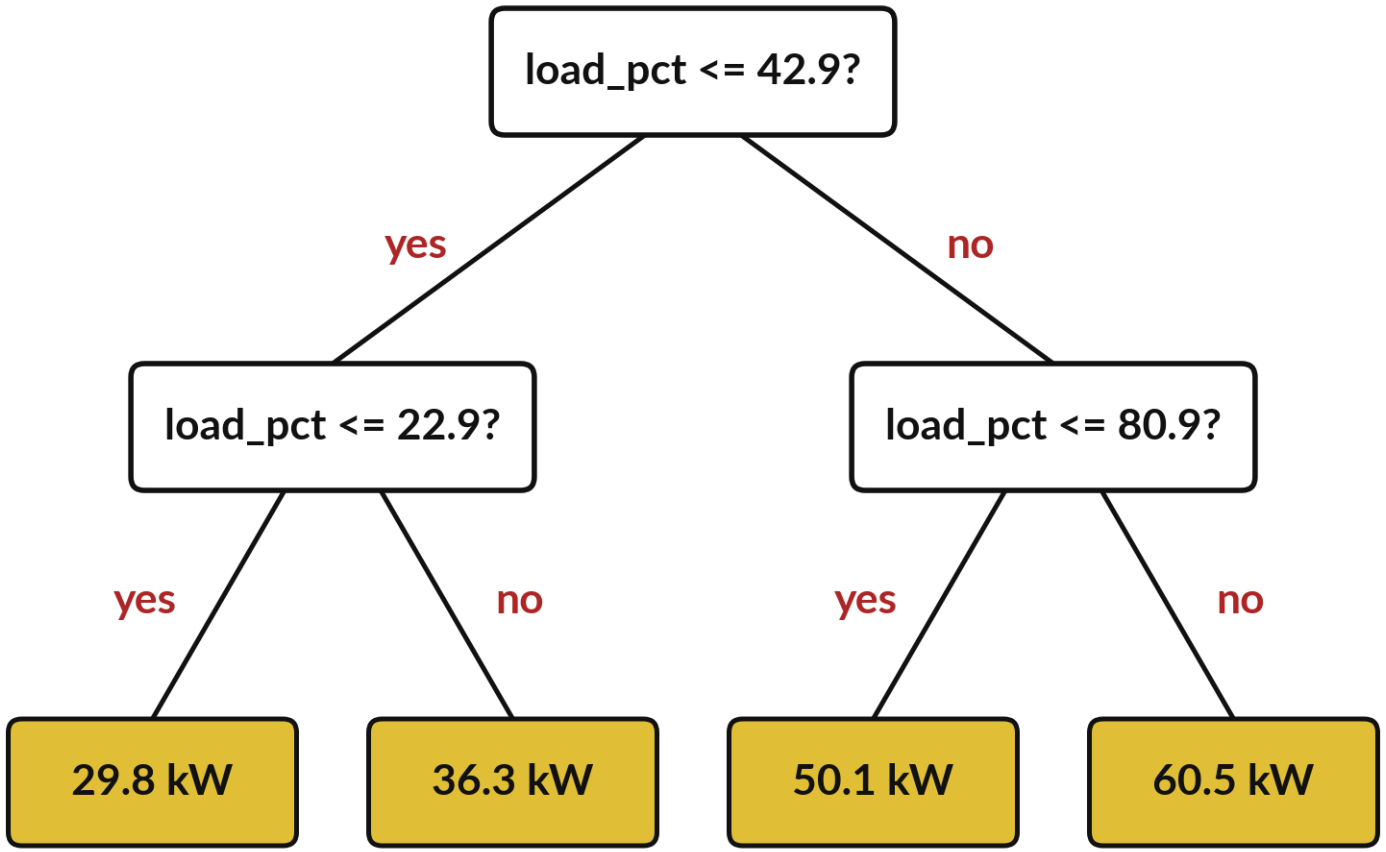
Each equipment group goes into one set only.

Imputation fills a missing value with the training median of its column



A model cannot use a missing value (empty cell, NaN). One training ambient_c value is missing. Medians: load_pct 62.8, ambient_c 26.1. Filling them is imputation (SimpleImputer).

A decision tree learns a short list of yes or no questions about the features

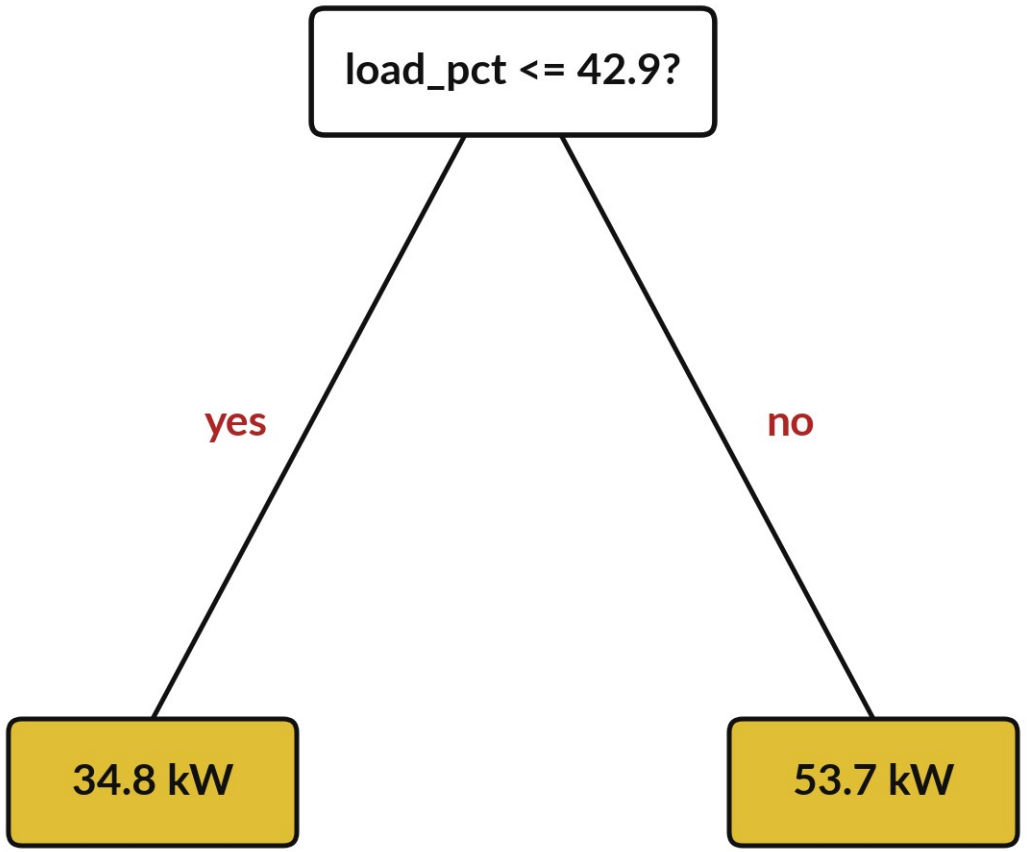


A row follows yes or no answers down to one yellow box, a leaf node. That leaf gives the prediction.

Each question is an internal node: a split on a feature and a threshold. The top one asks: is load_pct at most 42.9?

The prediction is the mean target of the training rows in that leaf node.

max_depth is the largest number of questions a row can be asked

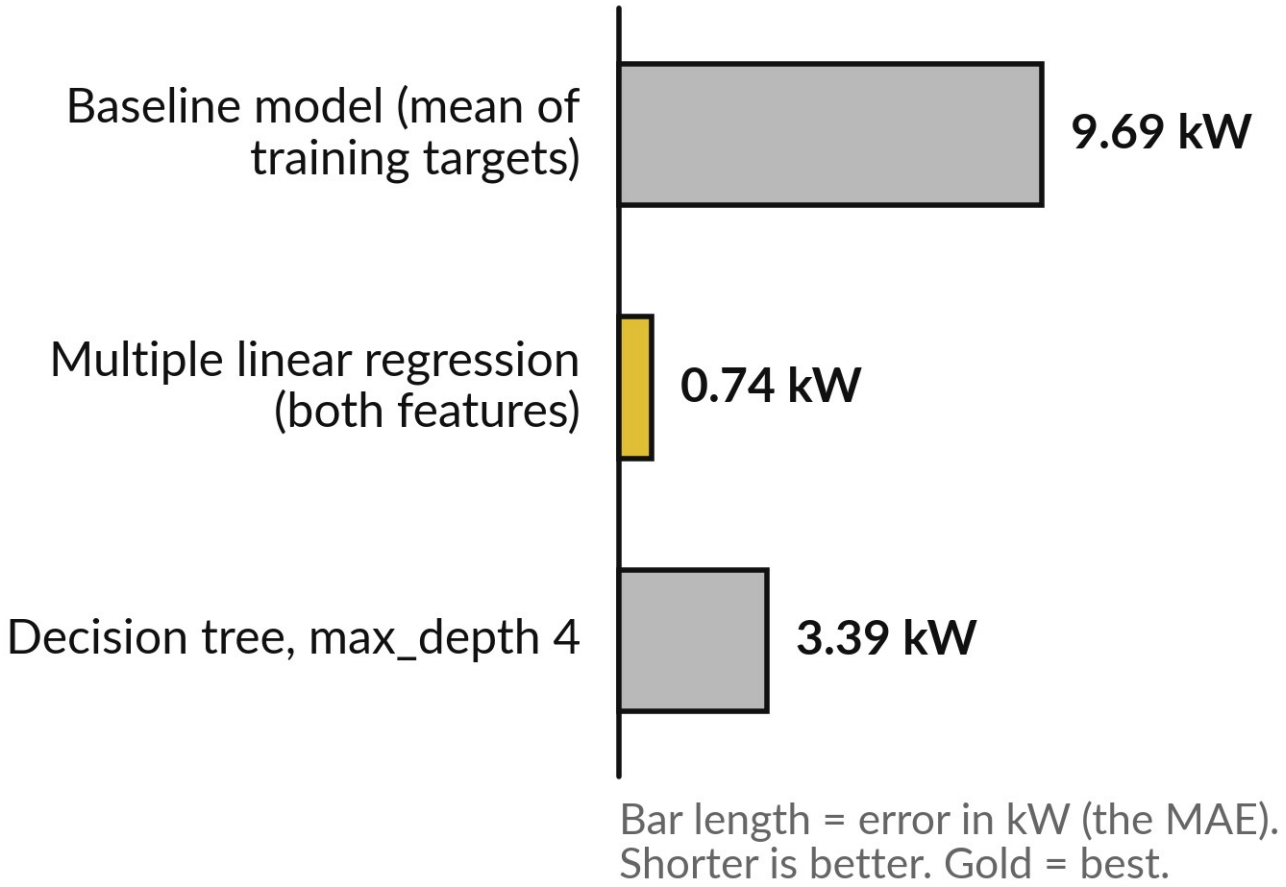


max_depth is a hyperparameter: a setting we choose before training. It limits how many questions a row meets.

This tree has max_depth 1: one question, two answers. The last tree had max_depth 2.

A deeper tree can fit the training set more closely, even overfit it (fit noise). Class 3 tests that. The lab uses max_depth 4.

Model comparison: three models evaluated on the same validation set



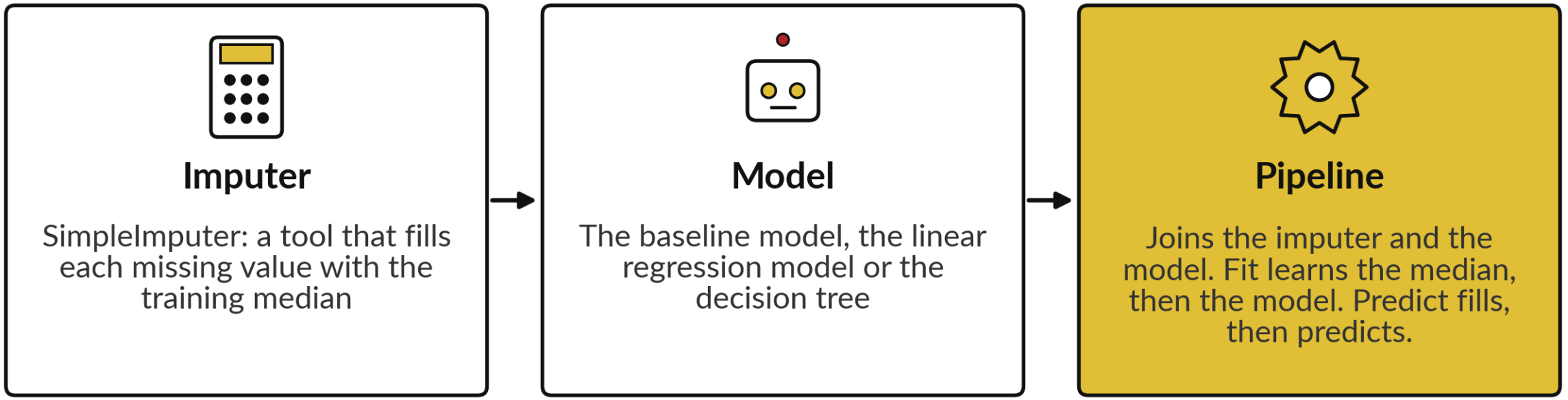
Multiple linear regression has a validation MAE of 0.74 kW. The baseline model has 9.69 kW.

Each bar is the MAE on the validation set.

Simple linear regression (load_pct only) scored 3.12 kW, not shown.

You produce these numbers in the lab.

A pipeline chains preprocessing and a model, and runs its steps in order



In the code that follows, the pipeline has two steps: the imputer first, then the linear regression model.

Read the workflow before you write it: create, fit, predict, score

```
imputer = SimpleImputer(strategy="median")
model = make_pipeline(imputer, LinearRegression())
model.fit(X_train, y_train)
y_pred = model.predict(X_valid)
mae = mean_absolute_error(y_valid, y_pred)
```

In plain words

Line 1: create an imputer (it fills missing values).

Line 2: make a pipeline: imputer, then linear regression.

Line 3, fit (training): learn the median, then b0, b1 and b2.

Line 4, predict (inference): y_pred holds the predicted validation targets.

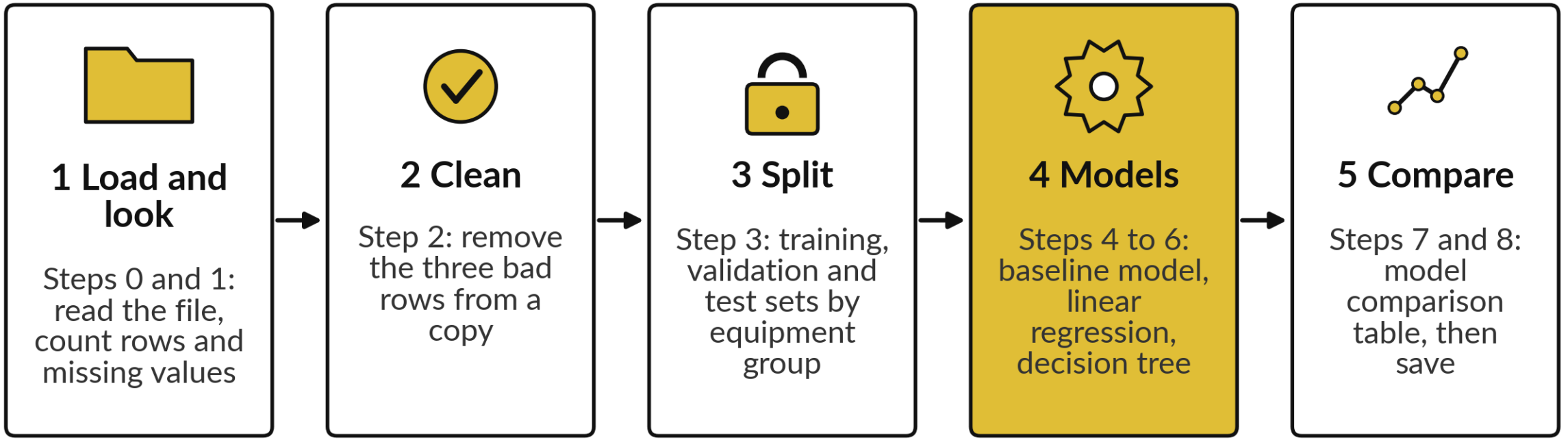
Line 5: score y_pred by the MAE.

X is a table of features. y is the target variable.

Part 2: the lab

Close the slides and open the lab notebook. Use the slides "Check your numbers" and "Four errors you will meet" at the end of this deck.

The lab has five stages, and the notebook numbers its steps 0 to 8



In every step: predict first, run the cell, read the output, then change one thing.

Check your numbers against these before you move on

What the notebook prints	You should see
Rows and columns of the raw file	91 rows, 5 columns
Rows left after removing the 3 bad rows	88
Rows in the training, validation and test sets	58, 15, 15
Baseline model: MAE on the validation set	9.69 kW
Simple linear regression (load_pct only): MAE (Step 5, CHANGE task)	3.12 kW
Multiple linear regression: MAE	0.74 kW
Multiple linear regression: intercept, coefficient of load_pct, coefficient of ambient_c	1.493, 0.447, 0.756
Decision tree, max_depth 4: MAE	3.39 kW

If your number differs, stop and ask. Do not change a number to match. Find the step that went wrong.

Four errors you will meet, what they mean and what to do

	Error message	What it says	What to do
	**FileNotFoundError**	The CSV is not beside the notebook	Upload engineering_data.csv in the Files panel
	**KeyError: target**	The code uses a column name that is not in the table, often a spelling difference	Print data.columns and copy the name exactly
	**NameError**	The code uses a name from a cell that has not been run yet	Run all cells from the top, in order
	**ValueError: contains NaN**	A missing value (an empty feature cell) reached the model	Fit the model inside the imputer pipeline, as in Step 5

How this class counts towards your coursework



Nothing is submitted today.



Block 1 is Classes 1 and 2. Its assignment has two parts: A7 (regression) and A8 (classification).



The Block 1 assignment closes on Tuesday 17 November, 11:00 (Kampala time).



The Block 1 quiz opens in the last 20 minutes of Class 2 and stays open for 24 hours.



Keep your saved Class 1 notebook for the assignment. Class 2 has its own notebook, and you reuse these cleaning steps.

Save the notebook and keep your three MAE scores

What you keep from today

Keep your lab notebook, saved after you restart the session and run all cells in order. Keep your three MAE numbers from the model comparison table: the baseline model, the multiple linear regression model and the decision tree.

Our question today

How can a computer predict cooling load from load and temperature, and how do we know it is any good?

Next class

Class 2: classification, where the model answers yes or no.

Take away

Saved notebook
Restart, run all, save

Model comparison
MAE of the three models

Next class
Classification

Bring
Laptop with the saved lab file